

Отчёт по лабораторной работе №6

Арифметические операции в NASM

Мещерякова София Андреевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
3	Выполнение заданий для самостоятельной работы	20
4	Выводы	23

Список иллюстраций

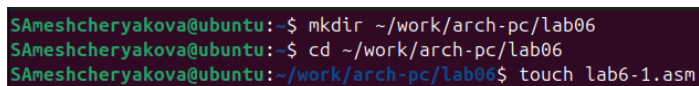
2.1	Создаем каталог и файл для лабораторной работы №6	5
2.2	Вводим текст программы	6
2.3	Запуск программы	7
2.4	Вводим текст программы	7
2.5	Запуск программы	8
2.6	Создание файла	9
2.7	Написание текста программы	9
2.8	Запуск программы	10
2.9	Изменённый текст программы	10
2.10	Запуск программы	11
2.11	Запуск программы	12
2.12	Создание файла	12
2.13	Текст программы	13
2.14	Запуск программы	14
2.15	Текст программы	15
2.16	Запуск программы	15
2.17	Создание файла	17
2.18	Написание программы	17
2.19	Запуск программы	18
3.1	Текст программы	22
3.2	Запуск программы при $x = 5$	22
3.3	Запуск программы при $x = 1$	22

1 Цель работы

Освоение арифметических инструкций языка ассемблера NASM.

2 Выполнение лабораторной работы

1. Создадим каталог для программ лабораторной работы № 6, перейдём в него и создадим файл lab6-1.asm(рис. 2.1).



```
$Ameshcheryakova@ubuntu:~$ mkdir ~/work/arch-pc/lab06  
$Ameshcheryakova@ubuntu:~$ cd ~/work/arch-pc/lab06  
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ touch lab6-1.asm
```

Рисунок 2.1: Создаем каталог и файл для лабораторной работы №6

2. Рассмотрим примеры программ вывода символьных и численных значений.
Программы будут выводить значения записанные в регистр eax.

Введем в файл lab6-1.asm текст программы из листинга 6.1 (рис. 2.2).

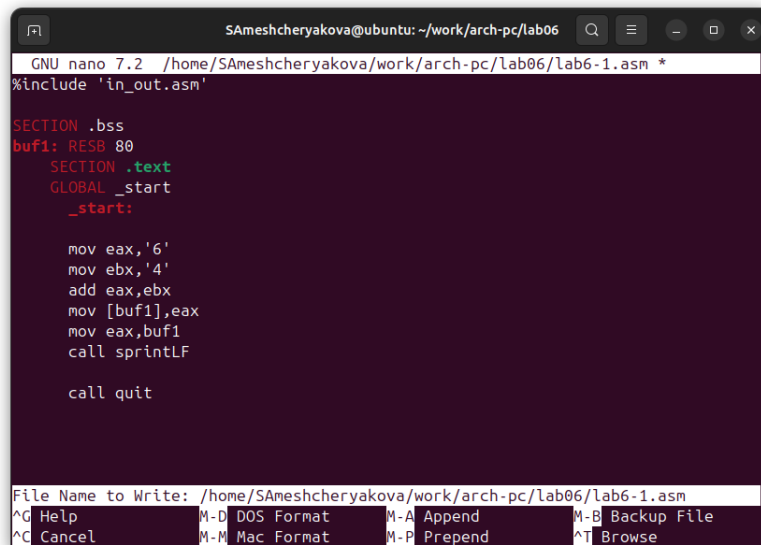


Рисунок 2.2: Вводим текст программы

Листинг 6.1

```
%include 'in_out.asm'

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:

    mov eax, '6'
    mov ebx, '4'
    add eax, ebx
    mov [buf1], eax
    mov eax, buf1
    call sprintf
```

call quit

Создадим исполняемый файл и запустим его (рис. 2.3).

```
SAmeshcheryakova@ubuntu: ~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
SAmeshcheryakova@ubuntu: ~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
SAmeshcheryakova@ubuntu: ~/work/arch-pc/lab06$ ./lab6-1
j
```

Рисунок 2.3: Запуск программы

3. Далее изменим текст программы и вместо символов, запишем в регистры числа. Исправим текст программы: заменим строковые значения на числовые (рис. 2.4).

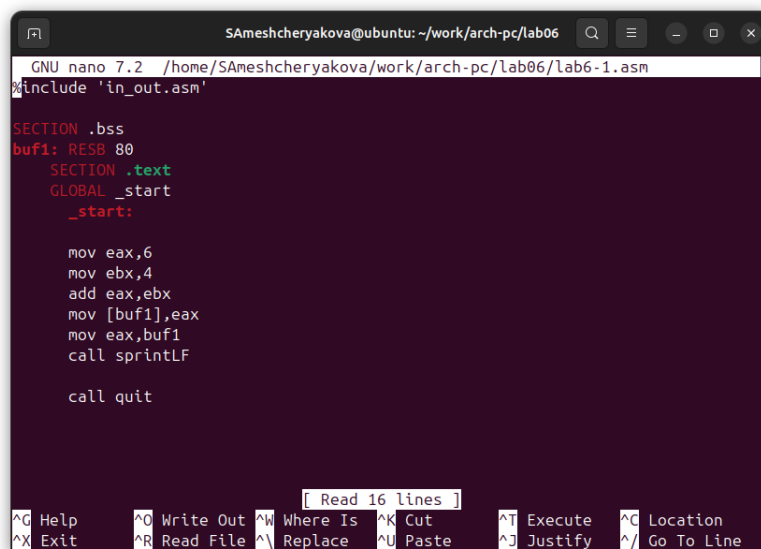


Рисунок 2.4: Вводим текст программы

Листинг 6.1 (с изменениями)

```
%include 'in_out.asm'
```

```
SECTION .bss
```

```

buf1: RESB 80

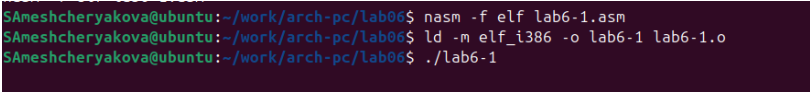
SECTION .text
GLOBAL _start
_start:

    mov eax, 6
    mov ebx, 4
    add eax, ebx
    mov [buf1], eax
    mov eax, buf1
    call sprintf

    call quit

```

Создадим исполняемый файл и запустим его (рис. 2.5).



```

$Aneshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
$Aneshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
$Aneshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-1

```

Рисунок 2.5: Запуск программы

Выводится символ с кодом 10, который не отображается на экране.

4. Создайте файл lab6-2.asm в каталоге ~/work/arch-pc/lab06 (рис. 2.7).

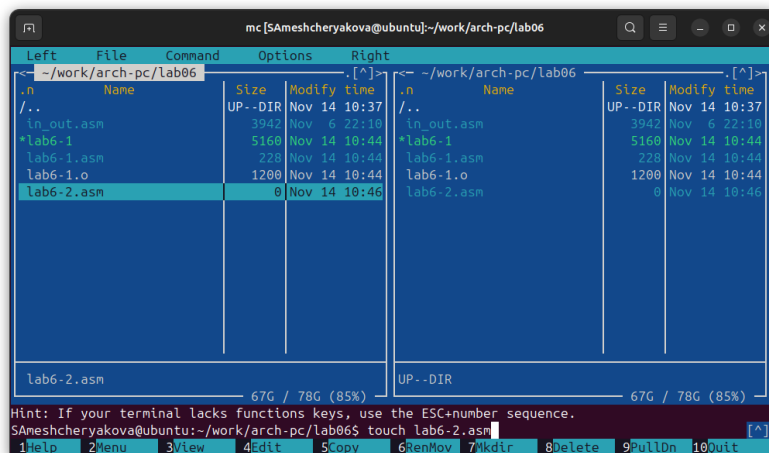


Рисунок 2.6: Создание файла

Введем в него текст программы из листинга 6.2 (рис. 2.7).

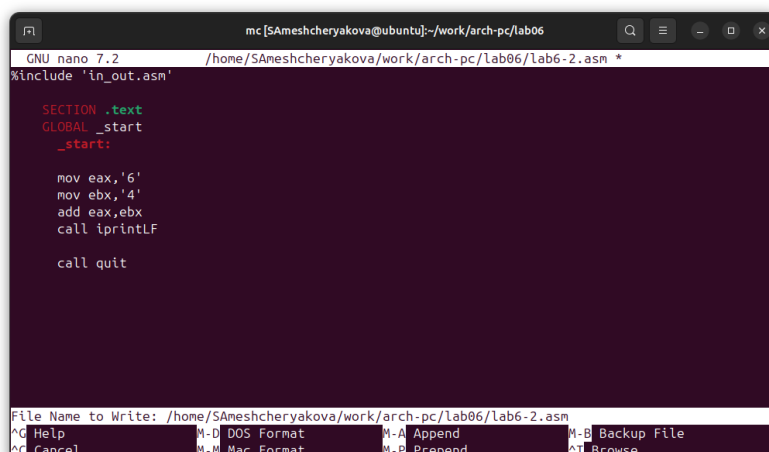


Рисунок 2.7: Написание текста программы

Листинг 6.2

```
%include 'in_out.asm'

SECTION .text
GLOBAL _start
```

```

_start:

mov eax, '6'
mov ebx, '4'
add eax, ebx
call iprintLF

call quit

```

Создадим исполняемый файл и запустим его (рис. 2.8).

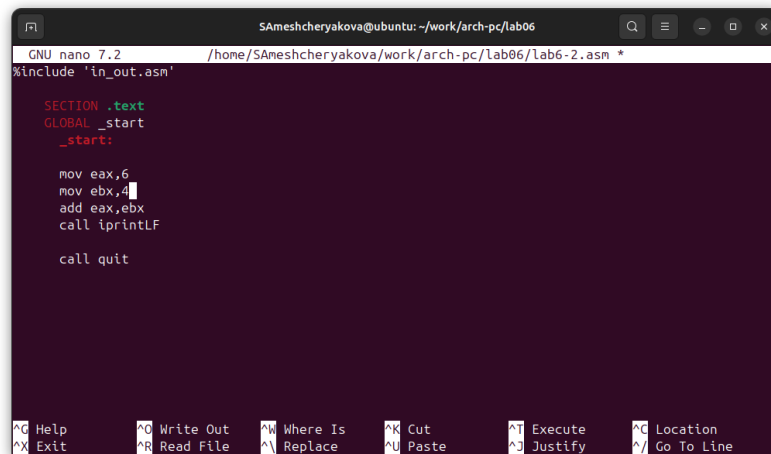
```

$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-2
106

```

Рисунок 2.8: Запуск программы

5. Аналогично предыдущему примеру изменим символы на числа. Замените строковые значения на числовые(рис. 2.9).



```

GNU nano 7.2 /home/$Ameshcheryakova/work/arch-pc/lab06/lab6-2.asm *
%include 'in_out.asm'

SECTION .text
GLOBAL _start
_start:

mov eax,6
mov ebx,4
add eax,ebx
call iprintLF

call quit

```

Рисунок 2.9: Изменённый текст программы

Листинг 6.2 (с изменениями)

```
%include 'in_out.asm'
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
mov eax,6
```

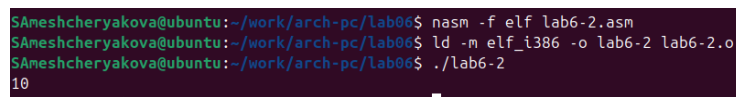
```
mov ebx,4
```

```
add eax,ebx
```

```
call iprintLF
```

```
call quit
```

Создадим исполняемый файл и запустим его (рис. 2.10).



```
SAneshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
SAneshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
SAneshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-2
10
```

Рисунок 2.10: Запуск программы

Результат запуска программы - число 10.

Заменяем функцию iprintLF на iprint.

Листинг 6.2 (с заменой iprintLF на iprint)

```
%include 'in_out.asm'
```

```
SECTION .text
```

```
GLOBAL _start
```

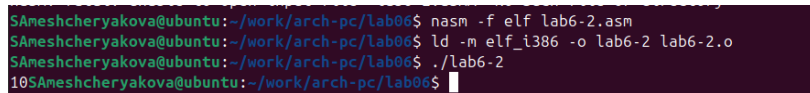
```
_start:
```

```
mov eax,6
```

```
mov ebx, 4
add eax, ebx
call iprint

call quit
```

Создадим исполняемый файл и запустим его (рис. 2.11).



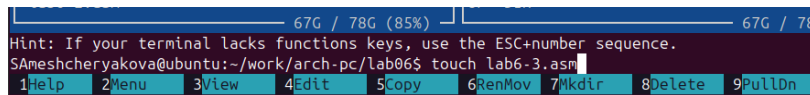
```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-2
10SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$
```

Рисунок 2.11: Запуск программы

В отличие от `iprintLF` функция `iprint` не добавляет символ перевода строки.

6. В качестве примера выполнения арифметических операций в NASM приведем программу вычисления арифметического выражения $f(x) = (5 * 2 + 3) / 3$.

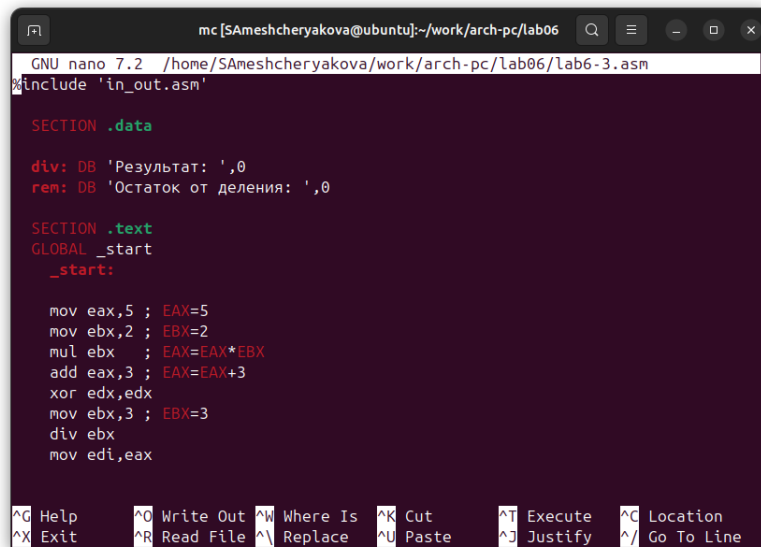
Создадим файл `lab6-3.asm` в каталоге `~/work/arch-pc/lab06` (рис. 2.12).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ touch lab6-3.asm
```

Рисунок 2.12: Создание файла

Введем текст программы из Листинга 6.3 (рис. 2.13).



```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab06
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab06/lab6-3.asm
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
_start:

mov eax,5 ; EAX=5
mov ebx,2 ; EBX=2
mul ebx ; EAX=EAX*EBX
add eax,3 ; EAX=EAX+3
xor edx,edx
mov ebx,3 ; EBX=3
div ebx
mov edi,eax

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^L Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Рисунок 2.13: Текст программы

Листинг 6.3

```
%include 'in_out.asm'

SECTION .data

div: DB 'XXXXXXXX: ',0
rem: DB 'XXXXXXXX XX XXXXXX: ',0

SECTION .text
GLOBAL _start
_start:

mov eax,5 ; EAX=5
mov ebx,2 ; EBX=2
mul ebx ; EAX=EAX*EBX
add eax,3 ; EAX=EAX+3
```

```

xor edx,edx

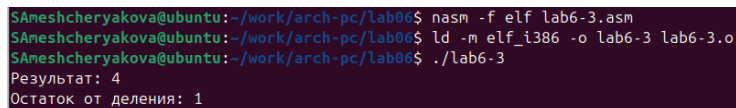
mov ebx,3 ; EBX=3

div ebx

mov edi,eax

```

Создадим исполняемый файл и запустим его (рис. 2.14).



```

$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1

```

Рисунок 2.14: Запуск программы

Изменим текст программы для вычисления выражения $f(x) = (4 * 6 + 2)/5$ (рис. 2.15).

Листинг 6.3 (для $f(x) = (4 * 6 + 2)/5$)

```

#include 'in_out.asm'

SECTION .data

div: DB 'XXXXXXXX: ',0
rem: DB 'XXXXXX XX XXXXXX: ',0

SECTION .text
GLOBAL _start
_start:

mov eax,4 ; EAX=4
mov ebx,6 ; EBX=6
mul ebx ; EAX=EAX*EBX
add eax,2 ; EAX=EAX+2

```

```

xor edx,edx
mov ebx,5 ; EBX=5
div ebx
mov edi,eax

```

```

GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab06/lab6-3.asm
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start
_start:

mov eax,4 ; EAX=4
mov ebx,6 ; EBX=6
mul ebx ; EAX=EAX*EBX
add eax,2 ; EAX=EAX+2
xor edx,edx
mov ebx,5 ; EBX=5
div ebx
mov edi,eax

```

Рисунок 2.15: Текст программы

Создадим исполняемый файл и проверим его работу (рис. 2.16).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1

```

Рисунок 2.16: Запуск программы

7. В качестве другого примера рассмотрим программу вычисления варианта задания по номеру студенческого билета, работающую по следующему алгоритму:

- вывести запрос на введение № студенческого билета
- вычислить номер варианта по формуле: $(S_n \bmod 20) + 1$, где S_n – номер студенческого билета (В данном случае $a \bmod b$ – это остаток от деления a на b).

- вывести на экран номер варианта.

Листинг 6.4

```
%include 'in_out.asm'

SECTION .data
msg: DB 'XXXXXXXX № XXXXXXXXXXXXXXXX XXXXX: ',0
rem: DB 'XXX XXXXXXXX: ',0

SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start
_start:

    mov eax, msg
    call sprintLF

    mov ecx, x
    mov edx, 80
    call sread

    mov eax, x
    call atoi

    xor edx, edx
    mov ebx, 20
```



```

div ebx
inc edx

mov eax, rem
call sprint
mov eax, edx
call iprintLF

call quit

```

Создадим файл variant.asm в каталоге ~/work/arch-pc/lab06 (рис. 2.17).

```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ touch variant.asm

```

Рисунок 2.17: Создание файла

Введем в файл текст программы из Листинга 6.4 (рис. 2.18).

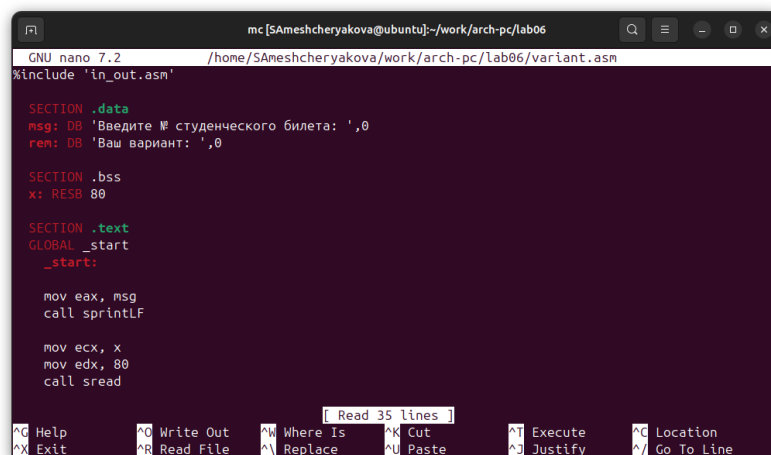


Рисунок 2.18: Написание программы

Создадим исполняемый файл и запустим его (рис. 2.19).

```

$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf variant.asm
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
$Ameshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032253634
Ваш вариант: 15

```

Рисунок 2.19: Запуск программы

Проверим результат выполнения программы.

- 1) $1032253634 \bmod 20 = 14$
- 2) $14 + 1 = 15$ - результат совпадает с полученным в программе.

2.0.1 Ответы на вопросы по программе из Листинга 6.4

1. Какие строки листинга 6.4 отвечают за вывод на экран сообщения „Ваш вариант:“?

За вывод этого сообщения отвечают строки

```

mov eax, rem
call sprint

```

2. Для чего используются следующие инструкции?

```

mov ecx, x      ; 1
mov edx, 80     ; 2
call sread     ; 3

```

- 1) загружает в регистр ecx адрес буфера x
- 2) mov edx, 80 — загружает в регистр edx число 80 (максимальное количество байт для считывания)
- 3) call sread — вызывает функцию sread из in_out.asm, которая считывает строку с клавиатуры, записывает её в буфер по адресу ecx, ограничивает длину edx байтами.

3. Для чего используется инструкция «call atoi»?

Инструкция call atoi вызывает функцию atoi из in_out.asm, которая позволяет преобразовать текстовое представление числа в целое число.

4. Какие строки листинга 6.4 отвечают за вычисления варианта?

За вычисления варианта отвечают строки:

```
xor edx, edx  
mov ebx, 20  
div ebx  
inc edx
```

5. В какой регистр записывается остаток от деления при выполнении инструкции «div ebx»?

Остаток от деления при выполнении div ebx записывается в регистр edx.

6. Для чего используется инструкция «inc edx»?

Инструкция inc edx увеличивает значение регистра edx на 1.

7. Какие строки листинга 6.4 отвечают за вывод на экран результата вычислений?

За вывод на экран результата вычислений отвечают строки:

```
mov eax, edx  
call iprintLF
```

3 Выполнение заданий для самостоятельной работы

1. Написать программу вычисления выражения $y = f(x)$. Для варианта 15 $f(x) = (5 + x)^2 - 3$, а с клавиатуры вводим значения $x_1 = 5$; $x_2 = 1$.

Листинг 6.5

```
%include 'in_out.asm'

SECTION .data
    msg: DB 'XXXXXX x: ',0
    div: DB 'f(x) = (5 + x)^2 - 3 = ',0

SECTION .bss
    rez: RESB 80
    x: RESB 80

SECTION .text
GLOBAL _start
_start:
    mov eax,msg
    call sprintfLF
```

```
mov ecx,x
mov edx,80
call sread

mov eax,x
call atoi

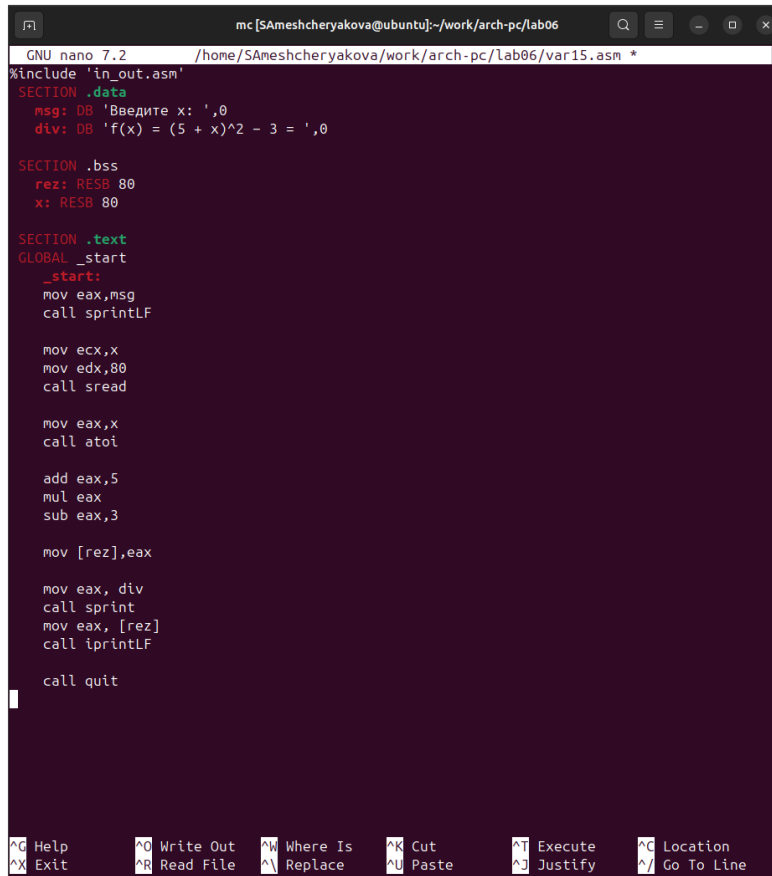
add eax,5
mul eax
sub eax,3

mov [rez],eax

mov eax, div
call sprint
mov eax, [rez]
call iprintLF

call quit
```

Создадим файл var15.asm, в котором напишем текст программы (рис. 3.1).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab06/var15.asm *
#include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
div: DB 'f(x) = (5 + x)^2 - 3 = ',0

SECTION .bss
rez: RESB 80
x: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax,msg
call sprintf

mov ecx,x
mov edx,80
call sread

mov eax,x
call atoi

add eax,5
mul eax
sub eax,3

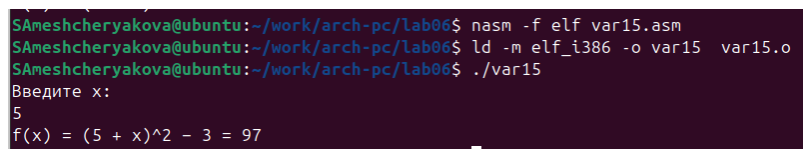
mov [rez],eax

mov eax, div
call sprintf
mov eax, [rez]
call iprintLF

call quit
```

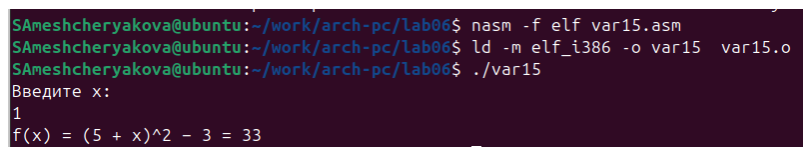
Рисунок 3.1: Текст программы

Создадим исполняемые файлы и запустим их (рис. 3.2, Рисунок 3.3).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf var15.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o var15 var15.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./var15
Введите x:
5
f(x) = (5 + x)^2 - 3 = 97
```

Рисунок 3.2: Запуск программы при $x = 5$



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ nasm -f elf var15.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ld -m elf_i386 -o var15 var15.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab06$ ./var15
Введите x:
1
f(x) = (5 + x)^2 - 3 = 33
```

Рисунок 3.3: Запуск программы при $x = 1$

4 Выводы

Мы приобрели навыки создания исполнительных файлов для решения выражений и освоили арифметические инструкции в NASM.